Distractor Filtering Through Open Knowledge Graph Concept Proximity for Dense RAG Systems

June 10, 2026

Gianluca Como (1605533)

Abstract

Dense retrieval can surface *distractors* — passages close to the query but answer-free — that degrade RAG accuracy, as Cuconasu et al. [3] show. We test whether *pure graph topology* on Wikidata can filter them, without using relation semantics. We build a 661.5M-edge entity graph from a local HDT dump, score query–document pairs by topological reachability (connected $\times$ purity), and fuse this with the dense score across a grid of hop-distance, hub-threshold and mixing weight α, evaluated on 1000 NQ-open queries with an LLM judge. The result is a clean *negative*: the KG signal never beats the dense baseline (0.364) beyond noise (best 0.372) and actively degrades accuracy as its weight grows (down to 0.290); 13/90 conditions are significant under McNemar+Bonferroni, all of them worse. We explain the mechanism (reachability saturates on a dense graph) and document the engineering and the methodological departures from [3].

1. Introduction

[3] show that high-scoring retrieved passages are often *distractors* — semantically similar to the query yet answer-free — and degrade the reader. We test one idea against this: filter distractors by *graph proximity* on an open knowledge graph, reranking retrieved passages by how close their entities are, on Wikidata, to the query entities. We deliberately restrict ourselves to pure topology — bare $\leq$ 3-hop reachability, no relation semantics — to isolate what proximity *alone* contributes. The answer is negative: topology alone does not filter distractors. The report's main contribution is this controlled test and the mechanistic account of *why* it fails; a by-product is a method to decide $\leq$ 3-hop reachability over the entire Wikidata graph without materialising hub neighbourhoods. Code {4}.

Email: Gianluca Como <comogianluca@gmail.com>.

Deep Learning and Applied AI 2025, Sapienza University of Rome, 2nd semester a.y. 2024/2025.

2. Related work

We adopt the corpus, retriever and reader of [3], who characterise the distractor problem in RAG. Retrieval uses Contriever over the gold-augmented Wikipedia corpus of [3]; entity linking uses the ReFinED linker of Ayoola et al. [1], one of the systems evaluated by Bast et al. [2]. Knowledge-graph approaches to RAG use the graph's relations and semantics; we instead use only its topology. The graph is stored as HDT over a Wikidata dump.

3. Method

3.1. Baseline: dense RAG

We reproduce the dense-retrieval pipeline of [3]. The corpus is its gold-augmented Dec-2018 Wikipedia release {1}; we re-segment it on sentence boundaries and pad each passage to exactly 100 words — the raw DPR passages have ragged, subject-less tails that hurt entity linking and the reader, and since Contriever mean-pools, fixed length is only a consistency choice. Passages are Contriever-encoded and indexed with FAISS (`IndexFlatIP`, ∼42M). From NQ-open {2} we keep short ($\leq$ 5-token) answers — matching the extractive setup of [3] — whose question and *all* answer variants link to Wikidata entities (ReFinED); we evaluate on a curated 1000-query subset (344/1000 queries are substituted so retrieved passages carry entities; see Appendix). The top-5 passages feed Llama-2-7B (base) under a one-shot extraction prompt, and correctness is decided by an *LLM judge* (Qwen2.5-7B-Instruct, a different model to avoid self-preference) that checks whether the response contains an NQ gold answer — replacing the brittle substring exact-match of [3] (prompt and judge in the Appendix).

3.2. Contribution: KG concept-proximity reranking

We rerank the top-100 by graph proximity on Wikidata. We query a pre-built HDT dump {3} (Jan 2022, ∼166 GB) via `pyHDT` — the public SPARQL endpoint timed out even

Table 1. LLM-judge accuracy, all 90 rerank settings (1000 NQ-open queries). Dense baseline = 0.364. Columns: KG weight α; rows: hop-distance k, hub-threshold t (∞=no filter; distance 1 has no bridge, so it is threshold-invariant). Best **bold**, worst underlined.

k	t	α=.1	.3	.5	.7	.9
1	any	.353	.344	.343	.316	.322
2	500	.362	.358	.361	.348	.354
2	1,000	.364	.361	.358	.343	.350
2	2,000	.360	.371	.357	.337	.341
2	5,000	.357	**.372**	.362	.337	.337
2	10,000	.364	.365	.357	.334	.329
2	∞	.354	.343	.311	.298	.295
3	500	.354	.344	.341	.329	.335
3	1,000	.346	.355	.342	.322	.310
3	2,000	.354	.349	.316	.303	.293
3	5,000	.366	.351	.310	.292	.290
3	10,000	.361	.362	.321	.306	.304
3	∞	.347	.333	.314	.314	.318

on a single hub count (see Appendix) — and export every direct entity-to-entity statement (Wikidata's "truthy" `wdt:` predicates), 6.61×10^8 edges. For a pair (q, d) we test $\mathrm{dist}(q, d) \leq 3$ *meet-in-the-middle*: distance 1 a set lookup, distance 2 an intersection of 1-hop neighbourhoods N_1, distance 3 an indexed edge-probe between $N_1(q)$ and $N_1(d)$ — never materialising a hub's neighbour list (`Q30`, *USA*: 2.35M edges). A degree threshold t drops high-degree bridges. The KG score is $s_{\mathrm{kg}} = \mathrm{cr} \cdot \mathrm{pr}$, where the *connected ratio* cr is the fraction of query entities reaching ≥ 1 document entity within k hops and the *purity ratio* pr the fraction of document entities reached by any query entity. We fuse with the dense score,

$$s = (1 - \alpha)\, s_{\mathrm{dense}} + \alpha\, s_{\mathrm{kg}}, \tag{1}$$

with α the *KG weight* and $s_{\mathrm{dense}} = \sigma / \max_q \sigma$ a per-query max-only rescaling of the raw Contriever score σ (the Appendix explains why not min–max). We sweep $k \in \{1, 2, 3\}$, $t \in \{500, 1\mathrm{k}, 2\mathrm{k}, 5\mathrm{k}, 10\mathrm{k}, \infty\}$, $\alpha \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$: 90 rerank cells plus the dense baseline.

4. Results

Baseline is sound. Dense retrieval scores 0.364, in line with the ≈0.36 that [3] report for Llama-2 with Contriever-retrieved documents on NQ-open — the harness is healthy and the comparison valid.

No gain, then harm. The best cell (α=0.3, dist 2, t=5k) reaches 0.372: +0.8pp ≈ 8 queries, i.e. noise (Table 1). As α grows the KG signal dominates and accuracy falls monotonically, bottoming at 0.290 (α=0.9, dist 3, t=5k).

It reshuffles, but does not help. The rerank is far from inert: at α=0.5 it rewrites the top-5 for >95% of queries (mean Jaccard@5 ≈ 0.12 vs. pure retrieval — barely one of five passages survives), yet accuracy stays at the baseline. It reorders aggressively without adding signal.

Threshold gradient is the mechanism. Holding α=0.9, dist 2, accuracy decays monotonically as the hub filter loosens, from 0.354 (t=500) to 0.295 (t=∞): admitting high-degree nodes as bridges connects everything to everything and destroys the signal.

Significance. Exact McNemar tests against the baseline, Bonferroni-corrected for the 90 comparisons (significance threshold $0.05/90 \approx 5.6 \times 10^{-4}$), flag 13 of the 90 conditions as significant — *all* worse, none better; each flips ≈200/1000 answers yet nets negative.

5. Discussion and conclusions

Why it fails. On a graph as dense as Wikidata, 2–3-hop reachability *saturates*: almost every document entity is reachable from almost every query entity, so cr ≈ 1, the score collapses onto pr alone, and discrimination is lost; the hub filter limits the damage but recovers no signal. For about 17% of queries the question entity is itself a very high-degree node (a country, a religion), where any passage is graph-close and the KG score is uninformative.

Limitations and future work. Our metric collapses relation semantics (`P31` *instance-of* and `P19` *place-of-birth* count alike), so it cannot screen out semantically irrelevant links. The honest next step is to use the graph's relations and semantics — not just its bare topology.

Conclusion. Pure topological proximity does not filter distractors for dense RAG on NQ-open; the value is the explained negative result and the reusable Wikidata tooling.

Data and code

{1} Corpus: https://huggingface.co/datasets/florin-hf/wiki_dump2018_nq_open.
{2} Queries: https://huggingface.co/datasets/florin-hf/nq_open_gold.
{3} Wikidata HDT dump: https://hdt-dumps.cluster.ai.wu.ac.at/dumps/.
{4} Code: delivery repository (to be created).

References

[1] Ayoola, T., Tyagi, S., Fisher, J., Christodoulopoulos, C., and Pierleoni, A. ReFinED: An efficient zero-shot-capable approach to end-to-end entity linking, 2022. URL https://arxiv.org/abs/2207.04108.

[2] Bast, H., Hertel, M., and Prange, N. A fair and in-

depth evaluation of existing end-to-end entity linking systems, 2023. URL https://arxiv.org/abs/2305.14937.

[3] Cuconasu, F., Trappolini, G., Siciliano, F., Filice, S., Campagnano, C., Maarek, Y., Tonellotto, N., and Silvestri, F. The power of noise: Redefining retrieval for RAG systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR 2024, pp. 719–729. ACM, 2024. doi: 10.1145/3626772.3657834. URL http://dx.doi.org/10.1145/3626772.3657834.

[4] Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., and Mian, A. A comprehensive overview of large language models, 2024. URL https://arxiv.org/abs/2307.06435.

[5] Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., and Tang, S. Graph retrieval-augmented generation: A survey, 2024. URL https://arxiv.org/abs/2408.08921.

Appendix

Pipeline: notebook flow

The project runs as nine notebooks in order (the Wikidata graph itself is built separately by `scripts/pipeline/`, Layers 1–3, under WSL2):

1. `01_corpus_preparation` — download the corpus and segment it into sentence-aligned 100-word passages (∼42M).
2. `02_nq_filtering` — filter NQ-open (≤5-token answers + ReFiNed entities) to 31,372 queries.
3. `03_embedding` — Contriever-encode every passage and build 9 FAISS shards.
4. `04_answer_preparation` — top-100 retrieval and passage entity-linking for the 1000-query subset.
5. `05_answer_curation` — find the 344 queries whose top-100 carry no entities and pick replacements.
6. `06_apply_curation` — apply the swap, producing the `*_curated` files.
7. `07_kg_rerank` — KG reachability (cr/pr) and the Jaccard reshuffle grid.
8. `08_llm_eval` — Llama-2-7B generates answers for all 91 conditions.
9. `09_llm_judge` — Qwen2.5 judges each answer; accuracy + McNemar.

Environment

Two environments are required:

- **Windows + `uv`** — everything except the graph build.
- **WSL2/Ubuntu** for the HDT pipeline: `pyHDT` needs a C++ build that does not compile cleanly on Windows native; the 166 GB dump is read via `/mnt/c`.
- **`faiss-gpu` via `conda`** — no Windows pip wheel, so it is installed in miniconda and bridged into the `uv` venv at runtime (`add_dll_directory` + `sys.path.insert`; conda paths are machine-specific).
- **ReFiNed patch** — ReFiNed V1 breaks on Windows + Python 3.12 + transformers 4.48 (a Unix-only `strftime("%s")` call); `scripts/tooling/patch_refined.py` fixes it, auto-run via subprocess.
- **HuggingFace token** in `.hf_token` at the repo root, for gated models (Llama-2).

All reported numbers are reproduced from the raw outputs by `documents/verify_report_numbers.py`; per-file schemas are catalogued in `DATA_DICTIONARY.md`.

Design choices

Dense normalization: max-only, not min–max. The raw Contriever score σ (inner product, not L2-normalised) sits in a narrow per-query band; we rescale by the per-query maximum only ($s_{\text{dense}} = \sigma / \max_q \sigma$). Min–max would stretch that band to $[0, 1]$ and change the ranking *philosophy*: e.g. A(σ=1.05, s_{kg}=0.2) vs B(σ=1.00, s_{kg}=0.4) at α=0.5 — max-only treats A's 5% dense edge as negligible and picks **B**, while min–max inflates that 5% gap to 100% and flips to **A**. Max-only preserves the dense geometry and only caps it at 1.

Reader prompt. Llama-2-7B base (not chat) is a completion model: without an exemplar it continues "Answer:" with prose. We prepend a one-shot extraction example and place the question *above* the documents (vs [3], which places it after); the top-ranked passage sits last, nearest the answer slot.

Judge. The judge must differ from the generator to avoid self-preference, so we use Qwen2.5-7B-Instruct (Apache-2.0, ungated), deciding YES/NO on whether a response contains a gold answer — absorbing paraphrase and formatting that a substring match misses.

What we tried first

The final reachability method is the third design.

SPARQL → local HDT. The public Wikidata SPARQL endpoint timed out even on a minimal `COUNT` over a hub

(`?x wdt:P31 wd:Q5`); a local HDT dump queried with `pyHDT` gives sub-millisecond predicate-restricted counts.

Wildcard $\rightarrow$ per-predicate export. A wildcard triple iterator overshot and silently dropped genuine Q–Q edges on a dump this large; exporting one predicate at a time gave exact, verifiable counts ($661{,}471{,}158$ edges).

BFS-N3 $\rightarrow$ meet-in-the-middle. Precomputing full 3-hop neighbourhoods by BFS timed out on 89% of queries (hubs explode the frontier); we instead precompute only 1-hop neighbourhoods ($\sim$93M rows) and decide $\leq$ 3-hop reachability per pair by meeting in the middle.

Query curation. 344/1000 subset queries had top-100 passages with no linkable entities (KG score then undefined); we substituted them with entity-bearing queries, which biases evaluation *towards* the KG — so the negative result reads "even on KG-favourable queries it does not help".